the browser will always include the cookie on requests, without looking at where the request comes from. The CSRF token fills this hole by serving as a marker that indicates where the request is coming from. It's active because the client making the request must manually add the token to the request. There is no way for the browser to automatically add the CSRF token to the request, or to even know what the value of the CSRF token is.

What is in a CSRF token?

We've talked about validating the CSRF token in abstract terms, but what exactly is inside the CSRF token that needs to be validated? Different servers implement CSRF tokens differently. The CSRF token in Express (from the CSURF package from https://github.com/expressjs/csurf) looks like this:

CSRF token = salt + crypto(salt + secret)

Here is what each of those pieces means:

- *Secret*—The secret is a per-server secret value. This can be set by the user in the session (which is important for coordinating secrets across servers, as you'll see later on), otherwise it will be randomly generated.
- *Salt*—The salt is another random value. But unlike the secret, the user cannot choose its value. The salt also has a fixed number of characters; at the time of this writing, Express's salt has 10 characters (the number of characters in the salt comes into play when validating the token).
- *Crypto*—The crypto function hashes the salt plus the secret using SHA1, and then base64 encodes the result. Hashing is a one-way operation that can't be reversed or decrypted.

Finally, the unencrypted salt value is prepended to the encrypted token value, and the sum is the CSRF token.

Let's look at an example of how to calculate the CSRF token. Suppose the server's secret is `SECRET`, and the salt is `0123456789`. The first step is to hash the value `0123456789SECRET` (the secret plus the salt). Suppose the result of this hash is `ABCDEF` (the hash value will look completely different from the secret and the salt). Finally, the salt is added to the hashed value, which is `0123456789ABCEDF`. This is the CSRF token.

When validating the token, the server doesn't decrypt the token and look at each part. (In fact, it can't decrypt the token, because hashing the token is a one-way operation that can't be reversed.) Instead, it looks at the incoming request CSRF token and grabs the first 10 characters. This is the salt value for this token. It then combines the salt value with the server secret to generate another token (using the same equation just noted). If the newly generated token matches the request CSRF token, the request is valid.

Turning again to the example, when the server receives the CSRF token `0123456789ABCEDF`, it first strips off the first 10 characters to get the salt, which is `0123456789`. Next, it runs the salt through the same equation. If this new value matches the CSRF token from the request, the request is valid.